МИНОБРНАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Отчет о выполнении лабораторной работы №3
по дисциплине «Архитектура суперкомпьютеров»

Исследование характеристик кэш-памяти компьютера

Студент группы 5130201/40101 _____ Гасов А.Е.
Преподаватель _____ Чуватов М.В.

«___» _____ 2026 г.

Санкт-Петербург
2026

1 Постановка задачи

Исследовать характеристики кэш-памяти собственного компьютера с помощью программы на языке C, используя косвенные измерения времени доступа к памяти. Необходимо:

1. Определить размер каждого уровня кэша (первого, второго, третьего) в килобайтах.
2. Определить размер строки кэша в байтах.
3. Определить степень ассоциативности кэша (современные кэши являются множественно-ассоциативными, k-way set associative).

2 Математическое описание

Кэш-память хранит копии данных основной памяти и делится на уровни (L1, L2, L3), каждый следующий из которых больше и медленнее предыдущего. Данные переносятся из памяти в кэш не по одному байту, а блоками фиксированного размера L - строками кэша. Время обращения к данным зависит от того, в каком уровне они находятся, поэтому по времени можно судить о характеристиках кэша, не обращаясь к документации процессора.

Среднее время обращения. В эксперименте измеряется полное время T выполнения N обращений к памяти, а среднее время одного обращения вычисляется как

$$t = \frac{T}{N}.$$

Деление на большое N необходимо, так как одно обращение занимает доли наносекунды и не может быть измерено системным таймером непосредственно.

2.1 Размер кэша

Пусть рабочий массив имеет размер S, а размер уровня кэша равен C. Если все элементы массива многократно обходятся, то при $S \leqslant C$ все данные находятся в этом уровне и t равно задержке этого уровня. При $S > C$ часть данных вытесняется и берется из следующего, более медленного уровня, поэтому t возрастает. Размером уровня считается наибольший S, при котором время еще соответствует этому уровню, за ним следует скачок. Для трех уровней в зависимости $t(S)$ ожидаются три плато, разделенные скачками.

2.2 Размер строки

Пусть массив обходится с шагом d байт и значительно превышает размер кэша, так что строка загружается из основной памяти. Пока $d < L$, в одну загруженную строку попадают L/d последовательных обращений, из которых только первое требует загрузки. При $d \geqslant L$ каждое обращение попадает в новую строку и требует загрузки. Следовательно, доля обращений с загрузкой равна $\min(1, d/L)$, и время $t(d)$ возрастает с ростом d до значения $d = L$, после чего перестает расти по этой причине. Размером строки считается шаг, на котором наблюдается наибольшее увеличение t относительно предыдущего шага.

2.3 Ассоциативность

Множественно-ассоциативный кэш (k-way set associative) разделен на наборы (sets), в каждом из которых k мест (путей, ways). Блок памяти может находиться только в одном определенном наборе, номер которого определяется адресом, но внутри набора занимает любое свободное место. При размере кэша C и строке L число наборов равно

$$m = \frac{C}{k \cdot L}.$$

Два адреса, различающиеся на величину, кратную C/k, попадают в один набор. Если взять n ячеек, расположенных с таким шагом, то все они борются за k мест одного набора. При $n \leqslant k$ все ячейки одновременно находятся в кэше и время t мало, а при $n = k+1$ хотя бы одна ячейка вытесняется, и t возрастает. Следовательно, ассоциативность равна наибольшему n, при котором время еще не увеличилось.

3 Особенности реализации

Программа реализована на языке C и компилируется с оптимизацией `-O2`. Все замеры основаны на обходе цепочки ячеек в массиве: в каждой ячейке хранится смещение следующей, а цепочка замкнута в кольцо. Параметры замера задаются константами: `ACCESSES_PER_RUN` - число обращений в одном замере, `RUN_COUNT` - число повторных замеров, по которым берется медиана. Переменная `result` (тип `volatile`) используется только для того, чтобы компилятор не удалил цикл обхода. Ниже описаны функции программы.

Функция time_seconds
Вход: состояние системных монотонных часов `CLOCK_MONOTONIC`
Выход: double - текущее время в секундах
Описание: Возвращает показание монотонных часов `CLOCK_MONOTONIC`, приведенное к секундам. Монотонные часы не зависят от изменения системного времени, поэтому разность двух показаний корректно задает длительность замера.

```
1 static double time_seconds(void) {
2     struct timespec ts;
3     clock_gettime(CLOCK_MONOTONIC, &ts);
4     return ts.tv_sec + ts.tv_nsec * 1e-9;
5 }
```

Листинг 1: time_seconds

Функция compare_doubles
Вход: const void *left, const void *right - указатели на два сравниваемых значения типа double
Выход: int: -1, 0 или 1 в зависимости от того, меньше, равно или больше первое значение второго
Описание: Функция сравнения для стандартной сортировки `qsort`. Используется для упорядочивания результатов повторных замеров перед выбором медианы.

```
1 static int compare_doubles(const void *left, const void *right) {
2     double x = *(const double *)left, y = *(const double *)right;
3     return (x > y) - (x < y);
4 }
```

Листинг 2: compare_doubles

Функция build_ring_by_memory_step
Вход: char *buffer - массив; size_t buffer_size - размер рабочей части массива в байтах; size_t memory_step_bytes - шаг между ячейками цепочки в байтах
Выход: изменяется содержимое массива `buffer`: в нем построена замкнутая цепочка ячеек
Описание: В ячейку по смещению i записывается смещение следующей ячейки $(i + memory_step_bytes) \bmod buffer_size$. Заполняются только ячейки, расположенные через шаг друг от друга. Операция взятия остатка замыкает цепочку в кольцо, что позволяет обходить массив неограниченное число раз.

```
1 static void build_ring_by_memory_step(char *buffer, size_t buffer_size, size_t
      memory_step_bytes) {
2     // заполняем только ячейки, которые будем посещать (по одной на каждые
      memory_step_bytes байт)
3     for (size_t i = 0; i < buffer_size; i += memory_step_bytes)
4         *(size_t *)(buffer + i) = (i + memory_step_bytes) % buffer_size;
5 }
```

Листинг 3: build_ring_by_memory_step

Функция build_ring_chaotic

Вход: char *buffer - массив; size_t buffer_size - размер рабочей части массива в байтах (степень двойки); size_t line_bytes - размер строки кэша в байтах

Выход: изменяется содержимое массива `buffer`: в нем построена замкнутая цепочка, посещающая все строки в хаотичном порядке

Описание: Массив делится на строки по $line_bytes$ байт, их число m равно $buffer_size/line_bytes$. В начале строки с номером i записывается смещение начала строки с номером $(5i + 1) \bmod m$. Так как m является степенью двойки, а числа 5 и 1 удовлетворяют условиям полного периода линейного конгруэнтного генератора, цепочка проходит по каждой строке ровно один раз. Порядок обхода не имеет постоянного шага, поэтому аппаратный префетчер не может предсказать следующий адрес.

```
1 static void build_ring_chaotic(char *buffer, size_t buffer_size, size_t line_bytes) {
2     size_t line_count = buffer_size / line_bytes;
3     // в начале каждой строки храним смещение начала следующей строки
4     for (size_t i = 0; i < line_count; i++)
5         *(size_t *)(buffer + i * line_bytes) = ((5 * i + 1) % line_count) * line_bytes;
6 }
```

Листинг 4: build_ring_chaotic

Функция walk

Вход: char *buffer - массив; size_t buffer_size - размер рабочей части в байтах; size_t memory_step_bytes - шаг цепочки (или размер строки для хаотичного порядка);
int isHaosStep - режим построения цепочки: 1 - хаотичный порядок, 0 - постоянный шаг

Выход: double - среднее время одного обращения в наносекундах (медиана по `RUN_COUNT` замерам)

Описание: Функция строит цепочку выбранным способом и выполняет прогрев: пять полных проходов по цепочке, чтобы данные закрепились в кэше. Затем `RUN_COUNT` раз выполняется замер: фиксируется время, выполняется `ACCESSES_PER_RUN` обращений вида `offset = *(size_t *)(buffer + offset)`, и разность времен делится на число обращений и переводится в наносекунды. Адрес каждого обращения берется из ячейки, прочитанной предыдущим обращением, поэтому процессор не может выполнять обращения параллельно, и измеряется именно задержка того уровня памяти, где находятся данные. Запись конечного смещения в переменную `result` объявленную как `volatile` не позволяет компилятору удалить цикл как не имеющий результата. Из результатов замеров выбирается медиана.

```
1  static double walk(char *buffer, size_t buffer_size, size_t memory_step_bytes, int
       isHaosStep) {
2      double run_times_ns[RUN_COUNT];
3      if (isHaosStep) build_ring_chaotic(buffer, buffer_size, memory_step_bytes);
4      else build_ring_by_memory_step(buffer, buffer_size, memory_step_bytes);
5      size_t offset = 0;
6      // прогрев: несколько полных проходов
7      for (size_t i = 0; i < 5 * (buffer_size / memory_step_bytes) + 1; i++)
8          offset = *(size_t *)(buffer + offset);
9      for (int run = 0; run < RUN_COUNT; run++) {
10         double start_time = time_seconds();
11         // следующий адрес читаем из только что прочитанной ячейки,
12         // поэтому процессор не может запустить обращения параллельно
13         for (long access = 0; access < ACCESSES_PER_RUN; access++)
14             offset = *(size_t *)(buffer + offset);
15         run_times_ns[run] = (time_seconds() - start_time) / ACCESSES_PER_RUN * 1e9;
16     }
17     result = offset;
18     // медиана: средний элемент отсортированного массива замеров
19     qsort(run_times_ns, RUN_COUNT, sizeof(double), compare_doubles);
20     return run_times_ns[RUN_COUNT / 2];
21 }
```

Листинг 5: walk

Функция main

Вход: параметров нет, используются константы `ACCESSES_PER_RUN` и `RUN_COUNT`, размеры массивов и шаги, заданные в коде

Выход: int - код завершения программы; в стандартный вывод печатаются три таблицы замеров и найденный размер строки

Описание: Выделяется массив максимального размера 256 МБ. Далее последовательно выполняются три эксперимента. В первом шаг цепочки увеличивается от 8 до 512 байт при размере массива 128 МБ; размером строки считается тот шаг, при переходе к которому время обращения возрастает относительно предыдущего шага сильнее всего. Во втором размер массива удваивается от 4 КБ до 256 МБ, цепочка строится в хаотичном порядке с найденным размером строки. В третьем берется k ячеек от 1 до 32 с шагом 256 КБ, которые попадают в один набор кэша.

```
int main(void) {
    // зададин максимальный размер массива в 256МБ
    size_t max_buffer_size = 256UL << 20;
    char *buffer = malloc(max_buffer_size);

    // 1. Размер строки
    printf("1. Размер строки\n");
    printf("шаг, байт    нс/обращение\n");
    size_t line_bytes = 0;
    double prev_time = 0, max_growth = 0;
    for (size_t memory_step = 8; memory_step <= 512; memory_step *= 2) {
        double time = walk(buffer, 128UL << 20, memory_step, 0);
        printf("%9zu    %8.3f\n", memory_step, time);
        // строка там, где время обращения вырастает сильнее всего
        if (prev_time > 0 && time / prev_time > max_growth) {
            max_growth = time / prev_time;
            line_bytes = memory_step;
        }
        prev_time = time;
    }
    printf("Размер строки: %zu байт\n", line_bytes);

    // 2. Размер кэша.
    // Размер строки берем из предыдущего замера.
    printf("\n2. Размер кэша\n");
    printf("размер, КБ    нс/обращение\n");
    for (size_t size = 4096; size <= max_buffer_size; size *= 2)
        printf("%10zu    %8.3f\n", size / 1024, walk(buffer, size, line_bytes, 1));

    // 3. Ассоциативность. Берем k элементов с шагом 256 КБ: такой шаг кратен размеру
    // одного "пути" кэша, поэтому все элементы попадают в один набор (set).
    // Пока k не больше числа путей, все помещаются. На k = ассоциативность + 1
    // одному не хватает места, и время растет.
    printf("\n3. Ассоциативность\n");
    printf("  k    нс/обращение\n");
    for (size_t elements = 1; elements <= 32; elements++)
        printf("%3zu    %8.3f\n", elements, walk(buffer, elements * (256UL << 10), 256UL
    << 10, 0));

    free(buffer);
    return 0;
}
```

Листинг 6: main

4 Результаты работы

Измерения выполнены на компьютере с процессором Apple M4 под управлением macOS. Привязка потока к ядру в macOS недоступна, поэтому планировщик мог выбрать любое из ядер. Вывод программы разбит на три таблицы по числу экспериментов и приведен в Листингах 7, 8, 9.

4.1 Размер строки

```
1. Размер строки
шаг, байт   нс/обращение
        8      0.294
       16      0.338
       32      0.991
       64      1.158
      128      9.324
      256     14.172
      512     29.137
Размер строки: 128 байт
```

Листинг 7: Определение размера строки

При шагах 8 и 16 байт время обращения составляет 0.29-0.34 нс, при шагах 32 и 64 байта - около 1 нс. При шаге 128 байт время возрастает до 9.3 нс, то есть в 8.1 раза по сравнению с шагом 64 байта. Это наибольший рост между соседними шагами, поэтому размер строки кэша составляет **128 байт**.

4.2 Размер кэша

```
2. Размер кэша
размер, КБ   нс/обращение
         4      0.917
         8      0.921
        16      0.920
        32      0.921
        64      0.922
       128      0.923
       256      5.260
       512      5.143
      1024      5.303
      2048      5.403
      4096      6.580
      8192      7.803
     16384     20.447
     32768     85.423
     65536     92.886
    131072     96.719
    262144     97.692
```

Листинг 8: Определение размера кэша

1. Для массивов от 4 до 128 КБ время обращения постоянно и составляет около 0.92 нс. При размере 256 КБ оно возрастает до 5.3 нс. Следовательно, размер кэша первого уровня составляет **128 КБ**.

2. Для массивов от 256 КБ до 8 МБ время держится в диапазоне 5.1-7.8 нс. При размере 16 МБ оно составляет 20.4 нс, а при 32 МБ - 85.4 нс. Для массивов от 64 МБ время выходит на значение порядка 90-98 нс. Следовательно, размер кэша второго уровня составляет **16 МБ**.

3. Третий уровень кэша в замерах не выявлен: после второго скачка новых ступеней нет.

4.3 Ассоциативность

```
3. Ассоциативность
  k   нс/обращение
  1       0.232
  2       0.922
  3       0.921
  4       0.921
  5       0.921
  6       0.921
  7       0.922
  8       0.921
  9       2.580
 10       3.725
 11       4.667
 12       6.542
 13       6.222
 14       6.929
 15       6.639
 16       6.921
 17       6.742
 18       6.996
 19       6.866
 20       6.704
 21       6.629
 22       6.369
 23       6.655
 24       7.022
 25       6.696
 26       6.353
 27       7.618
 28       6.624
 29       7.305
 30       6.536
 31       6.546
 32       7.116
```

Листинг 9: Определение ассоциативности

Для k от 2 до 8 ячеек время обращения постоянно и составляет около 0.92 нс. При $k = 9$ оно возрастает до 2.6 нс и далее растет до значения порядка 6.2-7.6 нс. Следовательно, в одном наборе кэша первого уровня помещаются 8 ячеек, то есть кэш **8-канальный множественно-ассоциативный** (8-way set associative). Значение при $k = 1$ (0.23 нс) в анализе не учитывается, так как цепочка из одной ячейки является особым случаем.

Заключение

В ходе выполнения лабораторной работы разработана программа на языке C, определяющая характеристики кэш-памяти компьютера по косвенным измерениям времени обращения к памяти. Для процессора Apple M4 получены следующие значения: размер строки кэша - 128 байт, размер кэша первого уровня - 128 КБ, размер кэша второго уровня - 16 МБ, ассоциативность кэша первого уровня - 8.

Реализовано:

1. Измерение среднего времени обращения к памяти по цепочке зависимых обращений с прогревом и выбором медианы по нескольким замерам.
2. Определение размера строки по зависимости времени от шага обхода, с автоматическим выбором шага, на котором время возрастает сильнее всего.
3. Определение размеров уровней кэша по зависимости времени от размера массива, с хаотичным порядком обхода строк для исключения влияния префетчера.
4. Определение ассоциативности по числу ячеек, конфликтующих за один набор кэша.

Список литературы

[1] The Apple Wiki. M4 [Электронный ресурс]. URL: https://theapplewiki.com/wiki/M4 (дата обращения: 29.10.2026).

[2] Hennessy J. L., Patterson D. A. Computer Architecture: A Quantitative Approach. 6th ed. Cambridge, MA: Morgan Kaufmann, 2017. 936 p. ISBN 978-0-12-811905-1.